Project Report
on
Chat with Retail Documents(RAG)

Course Name : Generative AI

Institution Name : Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Mohit Meharde	EN22CS301609
2	Nancy Agrawal	EN22CS301634
3	Niharika Panwar	EN22CS301645
4	Nikhil Garg	EN22CS301646
5	Mahima Sahu	EN22CS301570
6	Naman Mangroliya	EN22CS301628

Group Name : Group10D6

Project Number: GAI-21

Industry Mentor Name : Mr. Yash

University Mentor Name : Prof. Ajaj Khan

AcademicYear : 2025-2026

Table Of Content

Chapter No.	Chapter Name	Page No.
1	Problem Statement	3
1.1	Project Objectives	3
2	Scope of the Project	5
2.1	Functional Scope	5
2.2	Technical Scope	5
2.3	Out-of-Scope Elements	6
3	Proposed Solution	7
4	Key Features	8
4.1	Retail Document Classification	8
4.2	Recursive Text Chunking with Overlap	8
4.3	Semantic Vector Embedding and Persistent Storage	8
4.4	Context-Grounded Q&A with Source Citations	9
4.5	Interactive Web Interface	9
5	System Architecture	10
5.A	Presentation Layer	10
5.B	Application Layer	10
5.C	Data Layer	12
5.D	External Services Layer	12
5.E	Workflow Description	27
6	Tools & Technologies Used	16
6.1	Programming Language	16
6.2	Backend Framework	16
6.3	RAG Orchestration	16
6.4	Vector Database	16
6.5	AI & LLM Integration	17
6.6	Document Processing	17
6.7	Frontend	17
6.8	Configuration Management	17
7	Results & Output	18
7.1	Document Ingestion Results	18
7.2	Q&A Accuracy and Grounding	18
7.3	System Performance	18
7.4	Frontend Functionality	18
8	Conclusion	19
9	Future Scope & Enhancements	20
9.1	Multi-User Authentication and Role Management	20
9.2	Streaming LLM Responses	20
9.3	Multi-Format Document Support	20
9.4	Cloud Vector Database Migration	20
9.5	Conversational Memory	20
9.6	Automated Scheduled Re-Ingestion	20

1. Problem Statement

In the modern retail ecosystem, organizations generate and consume massive volumes of domain-specific documents, including product manuals, inventory reports, pricing catalogues, supplier agreements, and operational guidelines. Retail professionals must frequently access and analyze this information to support informed decision-making. However, manual search and retrieval from large collections of PDF documents is inefficient, time-consuming, error-prone, and lacks scalability.

Traditional keyword-based document search systems are limited in their ability to understand the semantic intent of user queries. As a result, they often return irrelevant results or fail to locate critical information embedded within lengthy and unstructured documents. Additionally, conventional question-answering systems based on general-purpose language models tend to produce hallucinated outputs—responses that appear plausible but are factually incorrect due to the absence of domain-specific grounding.

These limitations create significant challenges in extracting accurate, context-aware insights from retail documents, thereby impacting operational efficiency and decision quality. As the volume and complexity of retail data continue to grow, there is a clear need for more advanced and intelligent document processing solutions.

Therefore, there exists a need for an intelligent, scalable, and context-aware document intelligence system capable of:

- Ingesting and processing large volumes of retail PDF documents
- Performing semantic chunking and vector embedding of document content
- Retrieving the most contextually relevant information based on user queries
- Generating accurate and grounded responses using large language models
- Minimizing hallucinations by strictly relying on retrieved document context

The proposed **Retail Retrieval-Augmented Generation (RAG) System** addresses these challenges by integrating semantic vector search with large language model-based response generation. This approach ensures accurate, context-driven, and reliable information retrieval from domain-specific retail documents.

1.1 Project Objectives

The primary objective of the Retail Retrieval-Augmented Generation (RAG) System is to design and implement an intelligent document processing and query platform that enables retail professionals to upload domain-specific PDF documents and perform accurate, context-aware natural language question-answering over them.

The key objectives of the project are as follows:

- **Automated Document Ingestion:**
  Develop a robust pipeline for uploading, validating, and processing retail PDF documents, including text extraction and semantic chunking for downstream tasks.
- **Retail Domain Validation:**
  Implement an intelligent document classification mechanism using keyword analysis and semantic techniques to ensure only relevant retail-domain documents are accepted into the system.
- **Vector Embedding and Storage:**
  Generate high-dimensional semantic embeddings using advanced embedding models and store them efficiently in a vector database to enable fast and accurate similarity search.
- **Context-Aware Question Answering:**
  Enable natural language querying by integrating vector-based retrieval with large language models, ensuring responses are strictly grounded in the uploaded document corpus.
- **Interactive Web Interface:**
  Design a user-friendly single-page application (SPA) that allows seamless document management, query input, and display of answers with source citations for transparency.
- **System Reliability and Persistence:**
  Ensure persistent storage of vector data across system restarts, support incremental document ingestion, and provide mechanisms for clean updates or deletion of indexed data.

Collectively, these objectives aim to build a scalable and efficient retail document intelligence system that minimizes manual effort, enhances information retrieval accuracy, and provides traceable, context-driven insights for decision-making.

2. Scope of the Project

The scope of the Retail Retrieval-Augmented Generation (RAG) System defines the functional and technical boundaries of the proposed solution, along with the limitations of the current implementation.

2.1 Functional Scope

The functional scope outlines the core operations supported by the system. The platform includes:

- Upload and ingestion of retail-domain PDF documents
- Automated document validation using LLM-based classification (accept/reject mechanism)
- Text extraction, semantic chunking, and vector embedding of document content
- Storage and management of embeddings in a persistent vector database
- Natural language query submission and context-aware answer generation
- Retrieval of relevant document chunks using similarity search
- Display of responses with source citations (document name, page number, and content excerpt)
- Document management features including listing, deletion, and status tracking

The system also supports:

- Multi-file PDF upload in a single operation
- Conversational question-answering with session-based chat history
- Real-time backend health and vector database status monitoring
- User-friendly interface with theme support (Dark/Light mode) and preference persistence

2.2 Technical Scope

The technical scope defines the technologies and architectural components used in the system:

- Backend developed using Python with FastAPI framework and Uvicorn server
- Implementation of Retrieval-Augmented Generation (RAG) pipeline using LangChain
- Integration of Google Gemini models for document classification and response generation
- Use of embedding models for generating semantic vector representations
- ChromaDB as a persistent local vector database
- PDF processing using PyPDFLoader
- Text chunking using Recursive Character Text Splitter
- Frontend developed as a Single Page Application (SPA) using HTML, CSS, and JavaScript

The system is currently designed as a monolithic architecture but follows a modular approach, allowing future migration to a microservices-based architecture.

2.3 Out-of-Scope Elements

The following features are not included in the current version of the system:

- User authentication and role-based access control
- Multi-tenant document segregation
- Support for non-PDF file formats (e.g., DOCX, XLSX, images)
- Real-time streaming responses from LLMs
- Automated document re-ingestion or scheduled updates
- Fully distributed cloud-based vector database deployment

3. Proposed Solution

The proposed solution is an AI-powered Retail Document Intelligence System designed to automate document ingestion, semantic search, and context-aware question answering. The platform leverages a Retrieval-Augmented Generation (RAG) architecture to ensure accurate, grounded, and reliable responses derived strictly from domain-specific retail documents.

The system adopts a layered architectural approach to separate concerns and improve scalability, maintainability, and extensibility. The architecture is organized into the following layers:

- **Presentation Layer:**
  A user-friendly Single Page Application (SPA) developed using HTML5, CSS3, and JavaScript, served via FastAPI. This layer enables document upload, query submission, and visualization of responses with source citations.
- **Application Layer:**
  Handles core system logic, including document ingestion, validation, text chunking, embedding generation, and the RAG-based query processing pipeline. It also exposes REST APIs for frontend-backend communication.
- **Data Layer:**
  Consists of a ChromaDB vector database for storing semantic embeddings and a local file system for storing raw PDF documents. This layer ensures efficient retrieval and persistent storage.
- **External Services Layer:**
  Integrates with external AI services, including Google Gemini APIs for both embedding generation and large language model-based response generation.

Workflow Overview

The system operates through a two-phase workflow:

- **Ingestion Phase:**
  Users upload retail-domain PDF documents, which are validated using an LLM-based classification mechanism. The validated documents are processed through text extraction and split into manageable chunks.

  These chunks are then converted into high-dimensional vector embeddings and stored in the ChromaDB vector database for efficient retrieval.

- **Query Phase:**
  After user submits a natural language query, it is converted into an embedding and matched against stored vectors using similarity search.

4. Key Features

The Retail Retrieval-Augmented Generation (RAG) System incorporates advanced features that enable intelligent document management, accurate context-aware question answering, and reliable system performance.

4.1 Retail Document Classification

The system includes an intelligent document classification mechanism to ensure only relevant retail-domain PDFs are processed. It uses a dual-validation approach:

- **Keyword-based filtering:** Uses predefined retail terms such as product, SKU, inventory, pricing, etc.
- **LLM-based classification:** Generates structured output including classification status, confidence, and reason
- **Fallback mechanism:** Switches to keyword scoring if LLM is unavailable

This ensures high-quality and domain-specific document ingestion.

4.2 Recursive Text Chunking with Overlap

After validation, documents are processed using a recursive text chunking strategy to improve retrieval accuracy.

- Chunk size: ~1000 characters
- Overlap: ~200 characters (preserves context)
- Hierarchical splitting: paragraphs → sentences → words → characters
- Metadata stored: filename and page number

This approach ensures contextual continuity and precise source tracking.

4.3 Semantic Vector Embedding and Persistent Storage

The system converts text chunks into semantic vectors and stores them efficiently for fast retrieval.

- High-dimensional embeddings generated using embedding models
- Stored in ChromaDB vector database
- Persistent storage across server restarts
- Supports:
    - Initial indexing
    - Incremental updates
    - Full rebuild on deletion
    - Automatic reload on startup

This improves performance, scalability, and system reliability.

4.4 Context-Grounded Q&A with Source Citations

The core feature is a Retrieval-Augmented Generation (RAG) pipeline that ensures accurate and grounded responses.

- Converts user query into embedding
- Retrieves top relevant document chunks (similarity search)
- Sends context to LLM for answer generation
- Provides:
    - Accurate responses based only on documents
    - Fallback response if answer not found
    - Source citations (file name, page, excerpt)

This significantly reduces hallucination and improves trust.

4.5 Interactive Web Interface

The system provides a user-friendly Single Page Application (SPA) for seamless interaction.

- Features:
    - Document upload (multi-file support)
    - Conversational chat interface
    - Document management (view/delete)
    - Real-time system status
- UI enhancements:
    - Dark/Light mode with local storage
    - Chat history with response display
    - Source citation cards for each answer

This ensures a smooth, intuitive, and efficient user experience.

5. System Architecture

The system architecture of the Retail Retrieval-Augmented Generation (RAG) System is designed using a layered architectural model to ensure modularity, scalability, maintainability, and separation of concerns. The architecture is divided into four major layers: Presentation Layer, Application Layer, Data Layer, and External Services Layer. Each layer performs a specific function while interacting with other layers through well-defined interfaces.

This structured design ensures that changes in one component do not directly affect other components, thereby improving system stability and future extensibility.

A. Presentation Layer

The Presentation Layer serves as the user-facing interface of the system. It is implemented as a Single-Page Application using HTML, CSS, and JavaScript, served via the FastAPI backend.

This layer is responsible for:

• Accepting PDF uploads through drag-and-drop or file selection
• Submitting natural language queries to the system
• Displaying conversational responses with source citations
• Showing real-time system health and vectorstore status
• Managing documents (list, delete, refresh)

The interface includes:

• Chat-based interaction panel for Q&A
• Upload interface with multi-file support
• Document management dashboard
• Theme toggle (Dark/Light mode)

The Presentation Layer communicates with the Application Layer through REST APIs and does not directly interact with the database or external AI services, ensuring clean separation between UI and business logic.

B. Application Layer

The Application Layer acts as the core processing unit of the system. It is implemented using FastAPI and contains all business logic, processing pipelines, and API endpoints.

This layer includes the following components:

1. Document Ingestion Pipeline

The ingestion pipeline processes uploaded documents and prepares them for retrieval.

• Validates PDF format and retail domain relevance
• Extracts text using PDF processing tools
• Splits content into chunks (~1000 characters with overlap)
• Generates embeddings and stores them in the vector database

This ensures efficient and structured data preparation.

2. RAG Query Engine

The RAG engine handles query processing and response generation.

• Converts user query into embedding vector
• Performs similarity search to retrieve relevant chunks
• Constructs structured prompt with retrieved context
• Sends prompt to LLM for grounded response generation

This module ensures accurate and context-aware answers.

3. Vectorstore Manager

The vectorstore manager controls the lifecycle of the vector database.

• Initial creation of vector store
• Incremental addition of new documents
• Full rebuild on document deletion
• Automatic reload on system startup

It also maintains an in-memory reference for low-latency query processing.

4. API Endpoints

The system exposes RESTful APIs for communication:

• GET / – Serves frontend
• POST /upload – Handles document ingestion
• POST /chat – Processes queries
• GET /documents – Lists uploaded documents
• DELETE /documents/{filename} – Deletes documents
• GET /health – System status check

C. Data Layer

The Data Layer is responsible for storage and retrieval of all system data. It includes:

1. Vector Database (ChromaDB)

Stores semantic embeddings and metadata.

• Maintains document chunks as vectors
• Supports similarity search
• Persists data on local filesystem
• Enables fast retrieval

2. Raw PDF Storage

Stores uploaded documents for reference and reprocessing.

• Maintains original PDF files
• Used for rebuilding vector store
• Ensures data consistency

D. External Services Layer

The External Services Layer integrates third-party AI services required for processing.

1. LLM API (Gemini Chat API)

Used for:

• Retail document classification
• Context-grounded answer generation

Provides accurate and controlled responses.

2. Embedding API

Used for:

• Converting document text into vector embeddings
• Embedding user queries for retrieval

Enables semantic understanding and similarity search.

E. Workflow Description

The workflow of the Retail Retrieval-Augmented Generation (RAG) System is designed as a structured two-phase pipeline consisting of the Document Ingestion Phase and the Query-Response Phase. Each stage plays a critical role in ensuring accurate, efficient, and context-aware information retrieval.

Phase 1: Document Ingestion Workflow

Stage 1: PDF Upload

The workflow begins at the Presentation Layer, where the user uploads one or more PDF documents through the web interface. The frontend sends these files to the backend using a multipart/form-data request via the /upload API. Basic validation is performed to ensure that only PDF files are accepted.

Stage 2: Retail Domain Classification

Once uploaded, each document undergoes a validation process to ensure it belongs to the retail domain.

- Extracts initial content (first few pages)
- Applies **keyword-based filtering** using predefined retail terms
- Uses **LLM-based classification** for semantic validation
- Rejects non-relevant files with a reason

This step ensures that only domain-specific documents are processed further.

Stage 3: Text Chunking and Normalization

Accepted documents are processed for text extraction and segmentation.

- Full document text is extracted using PDF processing tools
- Split into chunks (~1000 characters each)
- Maintains overlap (~200 characters) for context preservation
- Metadata added: filename and page number

This step prepares the data for efficient embedding and retrieval.

Stage 4: Embedding and Vector Storage

Each text chunk is transformed into a semantic vector representation.

- Embeddings generated using embedding models
- Stored in ChromaDB vector database
- Metadata linked with each vector
- Data persisted for future queries

This enables fast similarity-based search during query processing.

Phase 2: Query-Response Workflow

Stage 5: Query Submission

The user submits a natural language question through the chat interface.

- Query sent to /chat API as JSON request
- Includes user question and parameters (e.g., include sources)

This initiates the retrieval and response generation process.

Stage 6: Query Embedding and Retrieval

The system processes the query to find relevant document content.

- Converts query into vector embedding
- Performs similarity search in vector database
- Retrieves top relevant chunks (e.g., top-5 matches)
- Includes metadata (file, page, content)

This ensures that only the most relevant context is used.

Stage 7: Prompt Construction and LLM Inference

The retrieved chunks are structured and passed to the language model.

- Formats chunks as labeled context (with source info)
- Combines context with user query
- Sends structured prompt to LLM

- Enforces rule: answer only from given context

This step ensures grounded and controlled response generation.

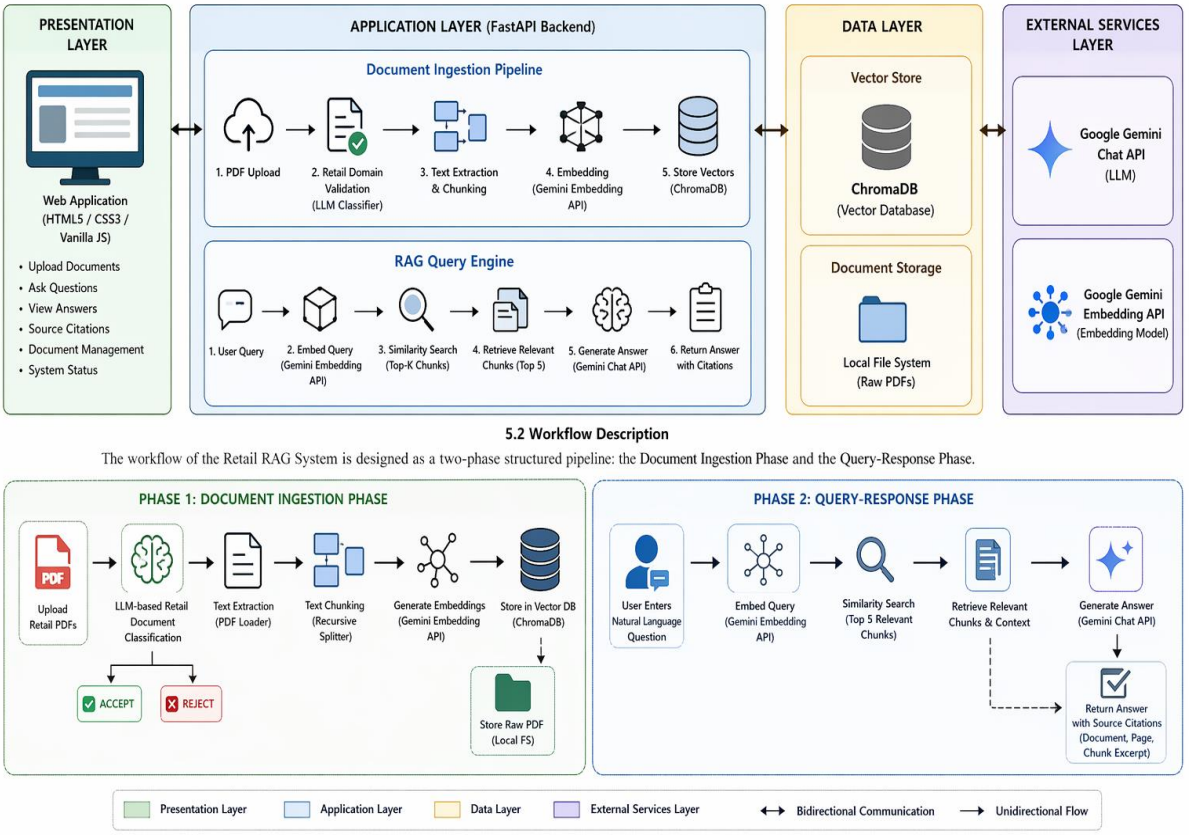
Stage 8: Response and Citation Display

The final response is generated and presented to the user.

- LLM returns context-based answer
- Includes fallback message if no answer found
- Response includes:
  - Answer text
  - Source citations (file name, page, excerpt)
- Displayed in chat interface with citation cards

This ensures transparency, accuracy, and user trust.

Figure 1: System Architecture Diagram – Retail RAG System



6. Tools &Technologies Used

6.1 Programming Language

6.1.1 Python 3.11+

Python is used as the primary backend development language due to its extensive ecosystem for AI integration, web framework support, and document processing libraries. It enables seamless interaction between the FastAPI server, LangChain orchestration layer, ChromaDB client, and Gemini API integrations.

6.2 Backend Framework

6.2.1 FastAPI

FastAPI is used as the REST API framework for its native asynchronous support, high performance, and automatic OpenAPI documentation generation. It manages all API endpoints including document upload, query handling, and system health monitoring. Uvicorn is used as the ASGI server for efficient request handling.

6.3 RAG Orchestration

6.3.1 LangChain

LangChain serves as the orchestration framework for the Retrieval-Augmented Generation pipeline. It provides tools such as PyPDFLoader for document ingestion, RecursiveCharacterTextSplitter for chunking, ChromaDB integration for vector storage, and prompt templates for structured LLM interaction. It allows modular and scalable pipeline design.

6.4 Vector Database

6.4.1 ChromaDB

ChromaDB is used as the embedded vector database for storing and querying document embeddings. It supports fast cosine similarity search and persists data locally, ensuring retrieval capability even after system restarts.

6.5 AI & LLM Integration

6.5.1 Google Gemini 2.5 Flash

Gemini 2.5 Flash is used for retail document classification and context-grounded answer generation. It ensures deterministic outputs with controlled prompting and reduces hallucination through context restriction.

6.5.2 Gemini Embedding-001

This model is used to generate 768-dimensional vector embeddings for both document chunks and user queries, enabling semantic similarity search within the vector database.

6.6 Document Processing

6.6.1 PyPDFLoader (pypdf)

PyPDFLoader is used to extract text from PDF documents page by page and convert them into structured document objects compatible with the LangChain pipeline.

6.6.2 RecursiveCharacterTextSplitter

This component splits text into manageable chunks using a hierarchical structure while maintaining contextual continuity through overlap.

6.7 Frontend

6.7.1 HTML5 / CSS3 / Vanilla JavaScript

The frontend is developed as a lightweight Single-Page Application using native web technologies. It handles user interaction, API communication, and dynamic UI updates without relying on external frameworks.

6.8 Configuration Management

6.8.1 python-dotenv

python-dotenv is used to securely manage environment variables such as API keys and configuration settings through a .env file, ensuring sensitive data is not hardcoded.

7. Results & Output

The Retail RAG System was successfully implemented and evaluated using multiple retail-domain PDF documents such as product catalogs, pricing reports, and inventory datasets. The system demonstrated efficient document ingestion, accurate classification, and reliable context-based question answering.

7.1 Document Ingestion Results

The system effectively classified retail documents using a dual-mode approach and rejected irrelevant files. The chunking process maintained context integrity, and embedding generation enabled smooth storage and retrieval.

• Accurate domain classification
• Efficient chunk generation with overlap
• Average ingestion time: 8–12 seconds per document

7.2 Q&A Accuracy and Grounding

The system produced highly accurate answers for queries related to product details, pricing, and inventory. The retrieval mechanism consistently selected relevant document chunks.

• Top-5 similarity retrieval ensured relevant context
• No hallucination due to strict context grounding
• Fallback responses for missing information

7.3 System Performance

The system demonstrated stable performance across ingestion and query operations with low latency retrieval and efficient API response handling.

7.4 Frontend Functionality

The frontend interface operated smoothly with all features functioning as expected.

• Drag-and-drop file upload
• Real-time system status updates
• Chat interface with "Thinking…" indicator
• Source citation display
• Persistent Dark/Light theme

8. Conclusion

The Retail RAG System provides an efficient solution for intelligent document processing by integrating PDF ingestion, semantic vector search, and LLM-based response generation into a unified architecture. It overcomes the limitations of traditional keyword-based search by delivering context-aware and citation-backed answers.

The system successfully achieves the following outcomes:

- Automated document ingestion and classification
- Semantic vector storage using ChromaDB
- Context-grounded Q&A with reduced hallucination
- Interactive and user-friendly frontend interface
- Reliable and scalable system architecture

The modular design ensures future scalability and easy integration of advanced features. Overall, the project demonstrates the practical application of Retrieval-Augmented Generation in retail analytics and knowledge retrieval systems.

9. Future Scope & Enhancements

The current system provides a strong foundation, but several enhancements can further improve its capabilities and scalability.

9.1 Multi-User Authentication and Role Management

Future versions can include secure login systems with role-based access control.

• User-specific document storage
• Admin and user role separation
• Secure session management

9.2 Streaming LLM Responses

Implementing real-time streaming responses can improve user experience by reducing perceived latency during long responses.

9.3 Multi-Format Document Support

Extend support beyond PDFs to include:

• DOCX, XLSX, CSV
• Image-based documents using OCR

9.4 Cloud Vector Database Migration

Migrating to cloud-based vector databases (e.g., Pinecone, Weaviate) can improve scalability and support distributed deployments.

9.5 Conversational Memory

Adding session-based memory will enable multi-turn conversations and improve contextual understanding of follow-up queries.

9.6 Automated Scheduled Re-Ingestion

Automating document updates will ensure the system always uses the latest data.

• Scheduled ingestion tasks
• Automatic vector updates